

Intro to DL4MSc: Attention and Transformer Blocks

Alexander Binder

December 13, 2025

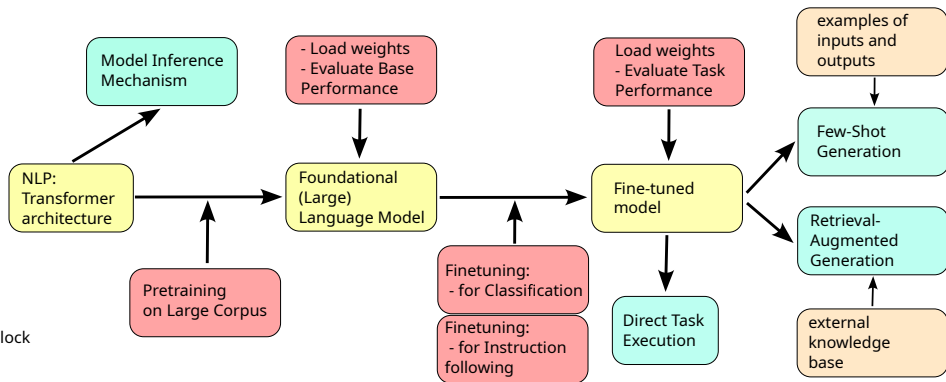
- 1 The high level plan + recap
- 2 Decoder: Key-value Caching for fast inference in sequential prediction
- 3 Decoder: The whole decoder transformer in code
- 4 Decoder: better inference than argmax
- 5 Encoder vs Decoder Architectures
- 6 BERT as encoder model
- 7 Finetuning examples for an Encoder

Data Preparation:

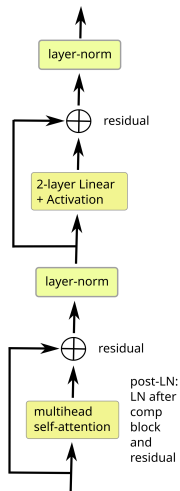
- Tokenization
 - Encoding
 - Decoding
- Dataset class
 - Minibatch Sampling

Arch:

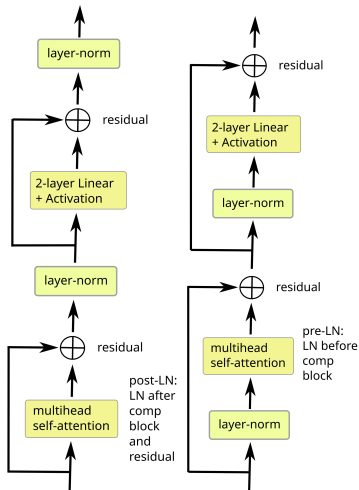
- Embedding Layer
- Attention Layer
- Decoder Transformer Block



We will build a network as a sequence of so-called **transformer decoder blocks**.

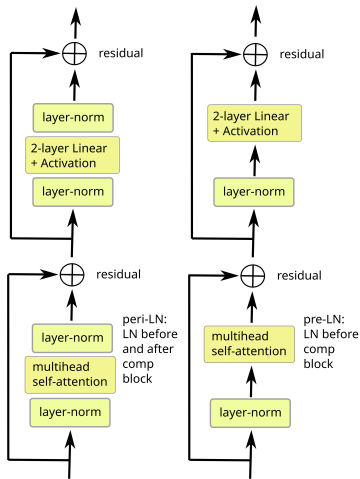


- important: MLP is applied for each token separately
- why MLP?
 $feature.shape = (bsize, seqlen, dim)$ applies the simplest non-linearity onto attention feature which mixed content across tokens and captured context from other tokens
- layernorm can be applied before the attention/mlp blocks (pre-LN) or after (post-LN) or sandwiching each block (peri-LN)



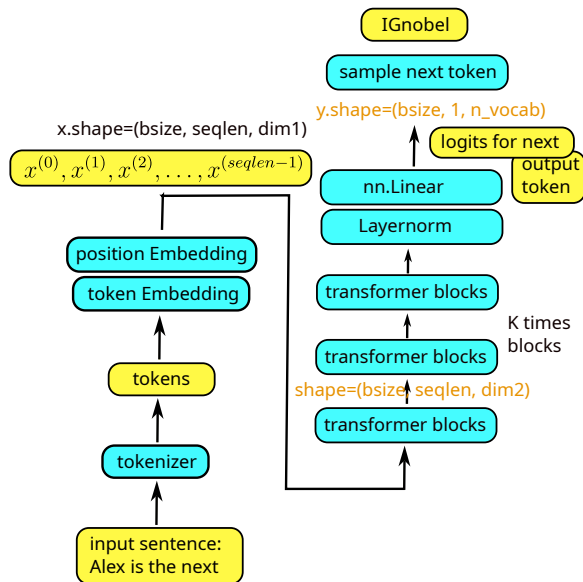
<https://arxiv.org/pdf/2002.04745>

- post-LN requires a learning rate warmup for training



Peri-LN <https://arxiv.org/html/2502.02732v1>

- 2x LN: before and right after each Attention and MLP block



Decoder for sequential generation

- ① The high level plan + recap
- ② Decoder: Key-value Caching for fast inference in sequential prediction
- ③ Decoder: The whole decoder transformer in code
- ④ Decoder: better inference than argmax
- ⑤ Encoder vs Decoder Architectures
- ⑥ BERT as encoder model
- ⑦ Finetuning examples for an Encoder

- common to use key-value caches in the attention layers of decoders (at inference time)

similarity matrix for a single element in the batch

its shape is: (seqlen,seqlen)

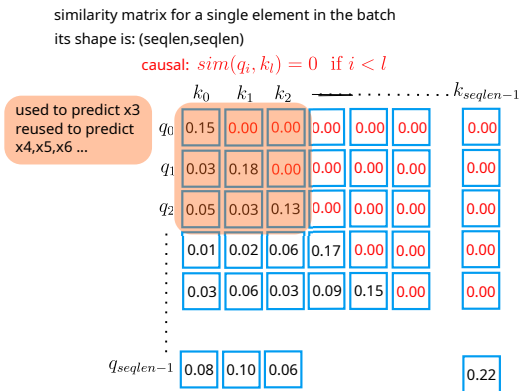
causal: $\text{sim}(q_i, k_l) = 0$ if $i < l$

used to predict x_2
reused to predict
 $x_3, x_4, x_5 \dots$

	k_0	k_1	k_2	...	$k_{\text{seqlen}-1}$
q_0	0.15	0.00	0.00	0.00	0.00
q_1	0.03	0.18	0.00	0.00	0.00
q_2	0.05	0.03	0.13	0.00	0.00
...					
...	0.01	0.02	0.06	0.17	0.00
...	0.03	0.06	0.03	0.09	0.15
...					
$q_{\text{seqlen}-1}$	0.08	0.10	0.06		0.22

- The 2x2 similarity matrix is used for an attention feature (for q_0, q_1), which is used to predict x_2 .
- The attention weighted feature computed for q_1 remains the same when k_2, k_3, k_4 is added - because all new key-value pairs result in zero-columns being added (on the row for q_1)!
- The attention feature computed for q_1 is reused - without any changes - to predict x_3, x_4, x_5 .
- when k_2 is added, a new row is added to the matrix (in flesh color) which computes a feature for q_2

- common to use key-value caches in the attention layers of decoders (at inference time)



- The 3x3 similarity matrix is used for an attention feature (for q_2), which is used to predict x_3 .
- The attention weighted feature computed for q_2 remains the same when k_3, k_4, k_5 is added - because all new key-value pairs result in zero-columns being added (on the row for q_2)!
- The attention feature computed for q_2 is reused - without any changes - to predict x_4, x_5, x_6 .
- when k_3 is added, a new row is added to the matrix (in flesh color) which computes a feature for q_3

- common to use key-value caches in the attention layers of decoders (at inference time)

similarity matrix for a single element in the batch
its shape is: (seqlen, seqlen)

causal: $\text{sim}(q_i, k_l) = 0$ if $i < l$

used to predict x4
reused to predict
x5, x6, x7 ...

	k_0	k_1	k_2	...	$k_{\text{seqlen}-1}$	
q_0	0.15	0.00	0.00	0.00	0.00	0.00
q_1	0.03	0.18	0.00	0.00	0.00	0.00
q_2	0.05	0.03	0.13	0.00	0.00	0.00
...	0.01	0.02	0.06	0.17	0.00	0.00
...	0.03	0.06	0.03	0.09	0.15	0.00
...						
$q_{\text{seqlen}-1}$	0.08	0.10	0.06			0.22

- can cache the weighted features $\sum_i w_i v_i$ from previous sequence elements in attention layers
- when a new sequence element is added, one needs to compute only one more row in the similarity matrix, and the resulting next weighted feature
- drastic speedup for inference (one needs to compute only one new weighted feature - for the newly added key,value pair) for each head of the attention layer

- 1 The high level plan + recap
- 2 Decoder: Key-value Caching for fast inference in sequential prediction
- 3 Decoder: The whole decoder transformer in code
- 4 Decoder: better inference than argmax
- 5 Encoder vs Decoder Architectures
- 6 BERT as encoder model
- 7 Finetuning examples for an Encoder

Taken from the Book by Sebastian Raschka. Think first of the parameters:

```
GPT_CONFIG_124M = {  
    "vocab_size": 50257,  
    "context_length": 1024,  
    "emb_dim": 768,  
    "n_heads": 12,  
    "n_layers": 12,  
    "drop_rate": 0.1,  
    "qkv_bias": False  
}
```

- "context_length" maximal sequence length

Create a code skeleton for the big picture first, which gets the interfaces and main compute layers right.

- ⦿ token embedding layer
- ⦿ position embedding layer
- ⦿ dropout for the embedding
- ⦿ N times transformerblock
- ⦿ final nn.Linear
- ⦿ one layernorm before the nn.linear
 - forward function calls them in the right order

Can instantiate it with dummy classes that just have the interfaces set

Taken from the Book by Sebastian Raschka, of note: `class nn.Sequential, *list`

```
import torch
import torch.nn as nn
class DummyGPTModel(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
        self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
        self.drop_emb = nn.Dropout(cfg["drop_rate"])
        self.trf_blocks = nn.Sequential( *[DummyTransformerBlock(cfg) for _ in range(cfg["n_layers"])] )
        self.final_norm = LayerNorm_v1(cfg["emb_dim"])
        self.out_head = nn.Linear(cfg["emb_dim"], cfg["vocab_size"], bias=False)

    def forward(self, in_idx):
        batch_size, seq_len = in_idx.shape
        tok_embeddings = self.tok_emb(in_idx)
        pos_embeddings = self.pos_emb(torch.arange(seq_len, device=in_idx.device))
        x = tok_embeddings + pos_embeddings
        x = self.drop_emb(x)
        x = self.trf_blocks(x)
        x = self.final_norm(x)
        logits = self.out_head(x)
        return logits
```

```
class DummyTransformerBlock(nn.Module):
    def __init__(self, cfg):
        super().__init__()
    def forward(self, x):
        return x

class LayerNorm_v1(nn.Module):
    def __init__(self, emb_dim):
        super().__init__()
        self.eps = 1e-5
        self.scale = nn.Parameter(torch.ones(emb_dim))
        self.shift = nn.Parameter(torch.zeros(emb_dim))
    def forward(self, x): # x.shape = (bsize, seqlen, hidden_dim)
        mean = x.mean(dim=-1, keepdim=True) # over the hidden dimension
        var = x.var(dim=-1, keepdim=True, unbiased=False) # over the hidden dimension
        norm_x = (x - mean) / torch.sqrt(var + self.eps) #element wise, stats computed over hidden dim
        return self.scale * norm_x + self.shift #a separate parameter for each element of hidden dim
```


next: components of one transformer block, and one transformer block

- ⊙ the MLP uses 1x GELU <https://docs.pytorch.org/docs/stable/generated/torch.nn.GELU.html>

```
class FeedForward(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(cfg["emb_dim"], 4 * cfg["emb_dim"]), #expansion inside
            nn.GELU(),
            nn.Linear(4 * cfg["emb_dim"], cfg["emb_dim"]),
        )
    def forward(self, x):
        return self.layers(x)
```

- pre-ln, post-ln, or peri-ln?
- components needed: 2x residual short cuts with a block-dropout for each of the blocks, causal multihead attention, mlp, 2 or 4 layer norms

is it pre-ln, post-ln, or peri-ln?

```
class TransformerBlock(nn.Module):
    def __init__(self, cfg):
        super().__init__()

        self.att = causal_multiheadattention(
            d_in=cfg["emb_dim"],
            d_out=cfg["emb_dim"],
            context_length=cfg["context_length"],
            num_heads=cfg["n_heads"],
            dropout=cfg["drop_rate"],
            qkv_bias=cfg["qkv_bias"])

        self.mlp = FeedForward(cfg)
        self.norm1 = LayerNorm(cfg["emb_dim"])
        self.norm2 = LayerNorm(cfg["emb_dim"])
        self.drop_block = nn.Dropout(cfg["drop_rate"])
```

```
class TransformerBlock(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        # [...]

    def forward(self, x):

        shortcut = x
        x = self.norm1(x)
        x = self.att(x)
        x = self.drop_block(x)
        x = x + shortcut

        shortcut = x
        x = self.norm2(x)
        x = self.mlp(x)
        x = self.drop_block(x)
        x = x + shortcut
        return x
```

real code is more complicated:

- key-value caching
- different methods to do inference with logits (not argmax over class indices!)
- options to turn on hardware-specific optimizations (e.g. use flash attention <https://arxiv.org/pdf/2205.14135> if available)

- ① The high level plan + recap
- ② Decoder: Key-value Caching for fast inference in sequential prediction
- ③ Decoder: The whole decoder transformer in code
- ④ Decoder: better inference than argmax
- ⑤ Encoder vs Decoder Architectures
- ⑥ BERT as encoder model
- ⑦ Finetuning examples for an Encoder

- known from multi-class classification in e.g. vision: train a model $f(x)$ which returns a vector of logits,
then use as predicted class $c(x) = \operatorname{argmax}_c f_c(x)$
- in generative tasks: makes created sequence deterministic given the first token $< \text{START} >$
undesired property!!
- goal: make sampling of sequences randomized

- ⊙ first idea: multinomial sampling:
 - $p_c(x) = \text{Softmax}(f(x))[c]$ defines a probability over all classes
 - draw a class c from the probability distribution

in code: <https://docs.pytorch.org/docs/stable/generated/torch.multinomial.html>

- ⊙ next: want to be able to make the probability distribution more sharp or more flat

- $p_c(x) = \text{Softmax}(f(x))[c]$ defines a probability
- rescale it with a temperature T :

$$p_c(x) = \text{Softmax}(f(x)/T)[c]$$

- draw from it
 - consider extreme limit cases $T \rightarrow \infty$, $T \rightarrow 0$

- $p_c(x) = \text{Softmax}(f(x))[c]$ defines a probability
- rescale it with a temperature T :

$$p_c(x) = \text{Softmax}(f(x)/T)[c]$$

- draw from it
 - consider extreme limit cases $T \rightarrow \infty$, $T \rightarrow 0$
 $T \rightarrow \infty$, $f(x)/T \rightarrow 0$
 $\text{softmax}(0, 0, \dots, 0) = 1/C$ therefore $p_c(x) \rightarrow 1/C$
 - physics intuition: higher temperature = more chaos

- drawback of plain sampling with a temperature: $T \rightarrow \infty$ increases probability of choosing alternatives, but it also allows to sample choices with very low probability under the original $T = 1$ -softmax.
- want a higher probability for alternatives, but only among the top-k, e.g. top-10 choices. two variants,
- variant 1:
 - fix k, select top-k classes $c_0, \dots, c_{k-1}$ using $f(x)$.
 - Set all other $f_c(x) = -\infty$. Apply softmax with temperature. ... Or directly apply the softmax only over the top-k subset
 - This will be equivalent to a softmax taken only over the top-k classes

$$\text{Softmax}(f_{c_0}(x)/T, f_{c_1}(x)/T, \dots, f_{c_k}(x)/T)$$

- effect allows more or less randomness, but restricted to top-k classes. No sampling of lower-ranked tokens.

- drawback of plain sampling with a temperature: $T \rightarrow \infty$ increases probability of choosing alternatives, but it also allows to sample choices with too low probability under the original $T = 1$ -softmax.
- want a higher probability for alternatives, but only among the top-k, e.g. top-10 choices. two variants,
- variant 2:
 - fix k as minimal number of classes selected, fix a (small) percentage R of probability covered
 - select the smallest number of top- m classes so that $\sum_{c=0}^{m-1} \text{softmax}(f_c(x)) \geq R$
This selects the smallest set of highest probability classes such that its cumulative probability exceeds a threshold R .
 - if $m < k$, select top- k
 - do softmax with temperature sampling among the selected classes as in Variant 1
- same: effect allows more or less randomness, but restricted to top- m classes. It ensures that we cover a certain percentage of probability for the highest ranked choices.

- known as quality-vs-diversity tradeoff.
 - Argmax is the max quality choice from the model viewpoint.
 - high temperature: more diversity ... but sampling of lower probability classes means lower quality of replies

- ① The high level plan + recap
- ② Decoder: Key-value Caching for fast inference in sequential prediction
- ③ Decoder: The whole decoder transformer in code
- ④ Decoder: better inference than argmax
- ⑤ Encoder vs Decoder Architectures
- ⑥ BERT as encoder model
- ⑦ Finetuning examples for an Encoder

Naming:

- Encoder: encode a sequence into a feature
- Decoder: decode a sequence from a feature

Decoder Architectures

- goal to generate a sequence one-by-one
- attention and network design is usually causal, that is a feature for sequence index i does not access information from features from future sequence indices $i + 1, i + 2, \dots$
- pretraining goal: loss for prediction of the next token
 - Let $x_0, x_1, x_2, \dots, x_{L-1}$ be the desired output sequence.
 - let $q_i(x)$ the softmax for logits of the i -th output .

Then the prediction model and training loss for the i -th output is

$q_i(x) = P(X_i | X[: i - 1] = (x_0, x_1, \dots, x_{i-2}, x_{i-1}))$ a vector of vocabulary size!

$$L_i(x) = \sum_{c \in \text{Vocab}} 1[x_i == c] (-1) \ln q_i(x)[c]$$

Use case: text generation one-by-one

Encoder Architectures:

- goal to process a sequence into a feature
- attention and network design is usually not causal. Can be bidirectional
- pretraining goal: loss for prediction of masked tokens in random positions of a sequence
 - Let $x_0, x_1, x_2, \dots, x_{L-1}$ be the desired output sequence.
 - let $q_i(x)$ the softmax for logits of the i -th output .
 - Then the prediction model and training loss for the i -th output is:

$f_i(x) = P(X_i | X[:i-1] = (x_0, \dots, x_{i-1}), X[i+1:] = (x_{i+1}, \dots, x_{L-1}))$ a vector of vocabulary size!

$$L_i(x) = \sum_{c \in \text{Vocab}} 1[x_i == c] (-1) \ln q_i(x)[c]$$

$$\text{Loss} = \sum_{i \in \text{masked}} L_i(x)$$

It is conditioned on the whole sequence except the one (or few) masked tokens

Encoder Architectures:

Use cases:

- prediction of global properties: toxicity, classification/ranking of texts for difficulty levels.
- Encoding of user data in general, e.g. for visual question answering, for text-guided image generation

Decoder Architectures

- inference goal: goal to generate a sequence one-by-one
- design: attention and network design is usually causal, that is a feature for sequence index i does not access information from features from future sequence indices $i + 1, i + 2, \dots$
- pretraining goal: loss for prediction of the next token, given all previous tokens

Encoder Architectures:

- inference goal: goal to process a sequence into a feature
- design: attention and network design is usually not causal. Can be bidirectional or use a global fusion (e.g. CNNs).
- pretraining goal: loss for prediction of masked tokens in random positions of a sequence

Encoder-Decoder Architectures, use cases:

- Translation: Encoder for source language sentence.
- instruction-following LLMs: Encoder for the input data and user commands.

- ① The high level plan + recap
- ② Decoder: Key-value Caching for fast inference in sequential prediction
- ③ Decoder: The whole decoder transformer in code
- ④ Decoder: better inference than argmax
- ⑤ Encoder vs Decoder Architectures
- ⑥ BERT as encoder model
- ⑦ Finetuning examples for an Encoder

Devlin et al. <https://aclanthology.org/N19-1423/> <https://arxiv.org/abs/1810.04805>

- inputs: concatenations of two(!) sentences, some tokens are masked

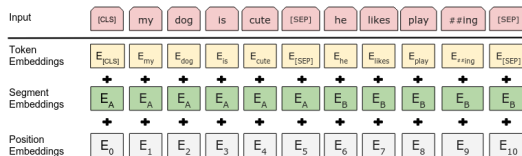


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

source: BERT paper <https://arxiv.org/abs/1810.04805>

- special token <SEP> to denote start of the second sentence.
- special token <MASK> to denote what is masked
- special token <CLS> to denote position where the feature is used for classification during finetuning (CLS is inputted during masked prediction training but not used to get classification vectors for masked token prediction. It is used to predict a separate task, see below.)

- inputs: concatenations of two sentences, some tokens are masked

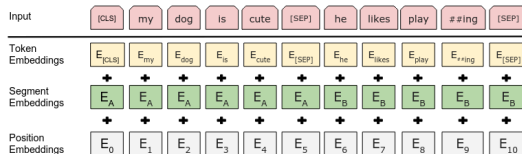
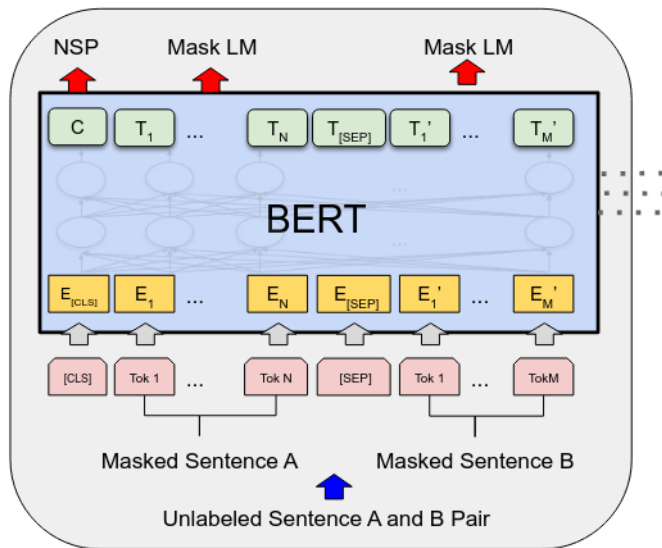


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

source: BERT paper <https://arxiv.org/abs/1810.04805>

- new: segment embeddings to denote whether a token belongs to the first or to the second of the two sentences.

Two pretraining tasks.



Two pretraining tasks.

- task 1: masked token prediction
- choose 15% of all tokens for masking. for a chosen token: $p = 0.8$ replace it with a <MASK> token, $p = 0.1$ replace it with a random word, $p = 0.1$ keep the word
- for prediction of the masked token, use feature on top of the transformer at the same position in the sequence as the token, classification task

Two pretraining tasks.

- task 2: so-called next sentence prediction. Question: Is the second sentence in the input – the next sentence in the original text corpus ? Binary classification problem
- remember the input are two sentences. $p = 0.5$ choose two adjacent sentences from the original text corpus . $p = 0.5$ choose a sentence, and another sentence from a random position.
- for prediction of the next sentence, use feature on top of the transformer at the position of the [CLS] token.

Input = [CLS] the man went to [MASK] store [SEP]
he bought a gallon [MASK] milk [SEP]
Label = IsNext

Input = [CLS] the man [MASK] to the store [SEP]
penguin [MASK] are flight ##less birds [SEP]
Label = NotNext

(## marks that both parts form one word)

The pretraining tasks and input format and training protocols matter as much for its success as the architecture!

- sequence of two sentences +[SEP] token as one input is intentional to be able model (question, answer) inputs during fine-tuning stages
- quote from the paper: For the pre-training corpus we use the BooksCorpus (800M words) (Zhu et al., 2015) and English Wikipedia (2,500M words). [...] It is critical to use a document-level corpus rather than a shuffled sentence-level corpus such as the Billion Word Benchmark (Chelba et al., 2013) in order to extract long contiguous sequences.
- section B.1 shows some fine-tuning tasks

The pretraining tasks and input format and training protocols matter as much for its success as the architecture!

- pretraining with a [CLS]-token and a classification task attached to it is common for encoders. Reason: they are finetuned often for classification tasks (see below.)

https://github.com/huggingface/transformers/blob/v4.57.1/src/transformers/models/bert/modeling_bert.py

https://github.com/huggingface/transformers/blob/v4.57.1/src/transformers/models/bert/configuration_bert.py

- ⊙ Vaswani et al. <https://arxiv.org/abs/1706.03762>
<http://nlp.seas.harvard.edu/annotated-transformer/>, use **only** its Encoder part
- ⊙ its encoder is a non-causal Post-LN transformer as seen in these two lectures
- ⊙ Important: Attention uses no mask at all, in particular no causal mask
(some implementation might implement it as one causal and one anticausal attention, but this is inefficient and not done so in Huggingface Transformers, see below)

```
class BertModel(BertPreTrainedModel): in
```

https://github.com/huggingface/transformers/blob/v4.57.1/src/transformers/models/bert/modeling_bert.py line 842 inits `class BertEncoder(nn.Module):`. This class uses `class BertLayer` with config, for `class BertLayer` we have

```
self.attention = BertAttention(config, layer_idx=layer_idx)
self.is_decoder = config.is_decoder
self.add_cross_attention = config.add_cross_attention
```

but config default is False for everything regarding cross-attention, decoders or encoder-decoders” see

```
class PretrainedConfig(PushToHubMixin): in
```

https://github.com/huggingface/transformers/blob/v4.57.1/src/transformers/configuration_utils.py

this implies that Bert uses encoder only!

How is the attention mask set ? See `def forward` of `class BertModel(BertPreTrainedModel):` .
This is there set as:

```
if attention_mask is None:
    attention_mask = torch.ones((batch_size, seq_length + past_key_values_length), device=device)
```

To understand what a 1 in `torch.ones` implies, check:

https://huggingface.co/transformers/v3.5.1/_modules/transformers/modeling_utils.html#ModuleUtilsMixin.get_extended_attention_mask which says: zero are entries to ignore and which does
`extended_attention_mask = (1.0 - extended_attention_mask) * -10000.0`

- ⦿ so zeros are mapped onto -10000.0 , ones are used
- ⦿ the default init is all ones:

```
if attention_mask is None:
    attention_mask = torch.ones((batch_size, seq_length + past_key_values_length), device=device)
```

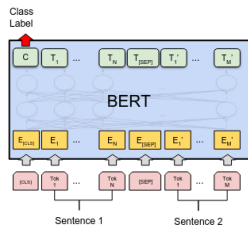
from the BERT paper:

We train with batch size of 256 sequences (256 sequences * 512 token [...]
for 1,000,000 steps, which is approximately 40 epochs over the 3.3 billion word corpus.

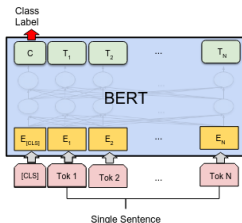
We use Adam with learning rate of $1e-4$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, L2 weight decay of 0.01, learning rate warmup over the first 10,000 steps, and linear decay of the learning rate. We use a dropout probability of 0.1 on all layers.

- ① The high level plan + recap
- ② Decoder: Key-value Caching for fast inference in sequential prediction
- ③ Decoder: The whole decoder transformer in code
- ④ Decoder: better inference than argmax
- ⑤ Encoder vs Decoder Architectures
- ⑥ BERT as encoder model
- ⑦ Finetuning examples for an Encoder

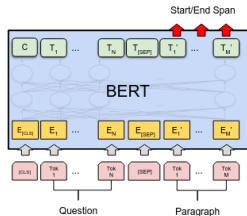
examples so that you have an idea what one can predict using sentences, or pairs of sentences :)



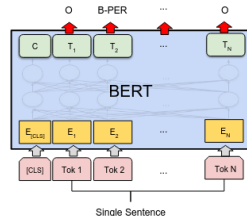
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA



(c) Question Answering Tasks:
SQuAD v1.1



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

instruction tuning (training data: pairs of (user inputs , assistant=LLM-replies)) will be done in a separate coding-homework

- QA - textual question answering with a multiple-choice answer as classification problem
- NLI - natural language inference. input: A premise sentence, a hypothesis sentence. Answer is : Hypothesis is true, is false, or cannot be inferred (entailment, contradiction, neutral). A 3-class classification problem.
- how to fine tune the encoder for classification problems ? It provides a final layer feature for every token position!
- the answer is different for NLI as above vs QA. Often the last layer feature over the [CLS] token is involved

some more from the GLUE Benchmark:

- MNLI Multi-Genre Natural Language Inference - an NLI task
- QNLI Question Natural Language Inference: (question, answer) pairs. Some contain the correct answer as part of the answer, others are sampled such that the correct answer is not included.
Binary Classification
- SST-2 The Stanford Sentiment Treebank: classify sentiment of movie reviews into 2 (originally 5) classes
- CoLA The Corpus of Linguistic Acceptability. Binary classification: is it a linguistically acceptable English sentence?
- **insight:** to classify, attach a prediction head over the [CLS] token feature vector. Works because the predicted property is not a token position.

some more from the GLUE Benchmark, based on prediction of similarity between pairs of sentences or paragraphs

- QQP Quora Question Pairs - two questions from quora are semantically equivalent or not ? Binary Classification
- STS-B The Semantic Textual Similarity Benchmark. 5 Score values to denote semantic similarity of two sentences.
- MRPC Microsoft Research Paraphrase Corpus: classification whether two sentences are semantically equivalent or not ? Binary Classification

- SQuAD 1.1 – text span prediction: have a pair of (question, answer) concatenated. Goal: predict start and end of the answer. We need to predict two token positions. How to model this ?
- at finetuning:
 - introduce 2 trainable vectors of dimensionality as the last BERT hidden layer: SPS for span start and SPE for span end . These act as prediction heads (nn.Linear ...) :
 - get for every input token x_i get its last layer feature $f(x_i)$.
 - compute probability to be a span start as $\text{softmax}_i(f(x_i) \cdot SPS)$, as span end: $\text{softmax}_i(f(x_i) \cdot SPE)$, use it with cross-entropy as binary classification problem.
 - ground truth class $\leftrightarrow$ position of start / end of answer in input

- ⊙ SQuAD 1.1 – text span prediction: have a pair of (question, answer) concatenated. Goal: predict start and end of the answer. How to model this ?
- ⊙ at inference time:
 - get for every input token x_i get its last layer feature $f(x_i)$.
 - To compute predicted start position and end position use the a score of $f(x_i) \cdot SPS + f(x_k) \cdot SPE$ based on the inner product of last layer token feature at i and j with the start and end features:

$$(\hat{i}, \hat{k}) = \operatorname{argmax}_{(i,k): i \leq k} f(x_i) \cdot SPS + f(x_k) \cdot SPE$$

- ⊙ SQuAD 2.0 – text span prediction: have a pair of (question, answer) concatenated. Goal: predict start and end of the answer. Allow for cases having no answer included in the answer part.
- ⊙ at finetuning:
 - same 2 trainable vectors: *SPS* for span start and *SPE* for span end as prediction heads.
 - if no answer, assume that the ground truth position for both heads (start and end) is the [CLS] token !

- ⊙ SQuAD 2.0 – text span prediction: have a pair of (question, answer) concatenated. Goal: predict start and end of the answer. Allow for cases having no answer included in the answer part.
- ⊙ at inference time:
 - get for every input token x_i get its last layer feature $f(x_i)$.
 - To compute predicted start position and end position use the a score of $f(x_i) \cdot SPS + f(x_k) \cdot SPE$ based on the inner product of last layer token feature at i and j with the start and end features:

$$(\hat{i}, \hat{k}) = \operatorname{argmax}_{(i,k): i \leq k} f(x_i) \cdot SPS + f(x_k) \cdot SPE$$

- compute no answer score as: $f([CLS]) \cdot SPS + f([CLS]) \cdot SPE$, where $f([CLS])$ is the feature over the [CLS] token, compare it to the best found pair $(\hat{i}, \hat{k})$ (with added threshold, determined on validation data)

Takeaway:

- how to use prediction heads for classification-type finetuning tasks ([CLS] token!)
- how to frame span prediction as a set of classification problems ... using two classification heads during training

<http://nlp.seas.harvard.edu/annotated-transformer/> for explanations of the original Vaswani et al. Encoder-Decoder

<https://arxiv.org/pdf/1910.10683> for more general architectures beyond encoder-decoder (prefixLM)

<https://aclanthology.org/2024.conll-babylm.24/>